

UNIT 3:
CPU SCHEDULING AND ALGORITHMS

Marks : 14

Hamid Ali

Lecturer, Computer Engineering

A.I. ARKP

CONTENTS

1. Scheduling Types [2hr]

1. Objectives
2. CPU and I/O burst cycles
3. Preemptive Scheduling
4. Non - Preemptive Scheduling
5. Scheduling Criteria

2. Types of Scheduling Algorithms [6hr]

1. FCFS, SJF, SRTN, RR, Priority, Multilevel queue scheduling.

3. Deadlock [2hr]

1. System Models, Necessary Conditions

4. Deadlock Handling

1. Prevention & Avoidance

Basic concept Scheduling

- CPU scheduling is a process used by the operating system to decide which task or process gets to use the CPU at a particular time. This is important because a CPU can only handle one task at a time, but there are usually many tasks that need to be processed. The following are different purposes of a CPU scheduling time.

Maximize the CPU utilization

Minimize the response and waiting time of the process.

- CPU Scheduling is the basis of multiprogrammed OS.
- By switching the CPU among processes, the OS can make the computer more productive.

CPU Burst and I/O Burst

CPU burst : A CPU burst is the period of time during which a process uses the CPU to execute instructions and perform calculations.

Example :A process runs on the CPU for 5 milliseconds, then waits for an I/O operation. That 5 ms is called a CPU burst.

I/O burst : An I/O burst is the period of time during which a process performs or waits for an Input/Output operation.

I/O (Input/Output) :

Reading from a disk

Writing to a file

Taking input from the keyboard

Sending data to a printer

CPU and I/O Burst Cycles

- The scheduling is a process in which OS takes the CPU away from one process and gives the CPU to another process.
- Process execution consist of a cycle of CPU execution and I/O wait. Processes alternate between these two states.
- Process execution begins with a CPU burst. That is followed by an I/O burst, which is followed by another CPU burst, then another I/O burst and so on.

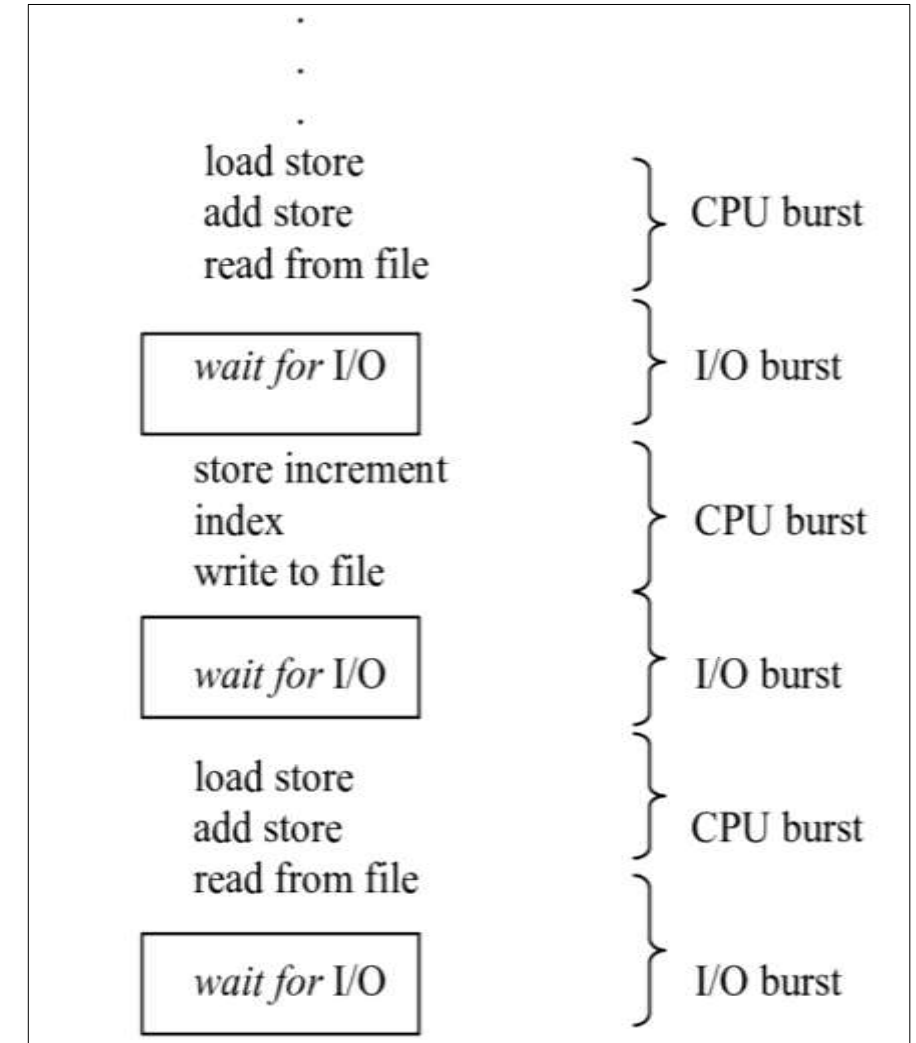


Fig: Alternating sequence of CPU and I/O burst

Scheduling Criteria (2m, 4m)

- CPU utilization
- Throughput
- Turnaround time
- Waiting time
- Response time
- Completion Time
- Priority

Scheduling Criteria (2m, 4m)

- **CPU utilization:** In multiprogramming the main objective is to keep CPU as busy as possible. CPU utilization can range from 0 to 100 percent.
- **Throughput:** It is the number of processes that are completed per unit time. It is a measure of work done in the system. Throughput depends on the execution time required for any process. For long processes, throughput can be one process per unit time whereas for short processes it may be 10 processes per unit time.
- **Turnaround time:** The time interval from the time of submission of a process to the time of completion of that process is called as turnaround time. It is the sum of time period spent waiting to get into the memory, waiting in the ready queue, executing with the CPU, and doing I/O operations.

Formula

Turn Around Time = Completion Time - Arrival Time.

Scheduling Criteria (2m, 4m)

- **Waiting time:** It is the sum of time periods spent in the ready queue by a process. When a process is selected from job pool, it is loaded into the main memory (ready queue). A process waits in ready queue till CPU is allocated to it. Once the CPU is allocated to the process, it starts its execution and if required request for resources. When the resources are not available that process goes into waiting state and when I/O request completes, it goes back to ready queue. In ready queue again it waits for CPU allocation.

Formula:

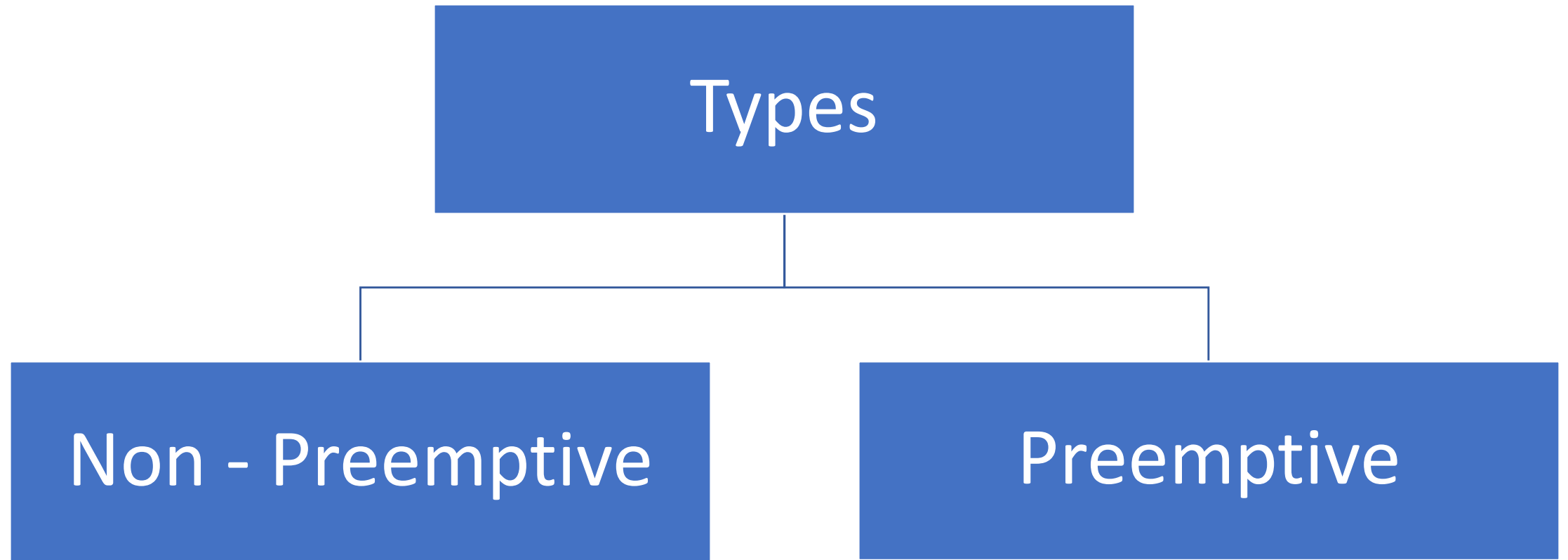
Waiting Time = Turnaround Time - Burst Time.

- **Response time:** The time period from the submission of a request until the first response is produced is called as response time. It is the time when system responds to the process request not the completion of a process. In the system, a process can Produce some output fairly early and can continue computing new results while previous results are being output to the user.

Scheduling Criteria (2m, 4m)

- **Completion Time:** The completion time is the time when the process stops executing, which means that the process has completed its burst time and is completely executed.
- **Priority:** If the operating system assigns priorities to processes, the scheduling mechanism should favor the higher-priority processes.

Types of Scheduling



Preemptive Scheduling

In preemptive scheduling, the operating system can interrupt a running process to allocate the CPU to another process usually due to priority rules or time-sharing policies. A process may be moved from Running → Ready state before it finishes.

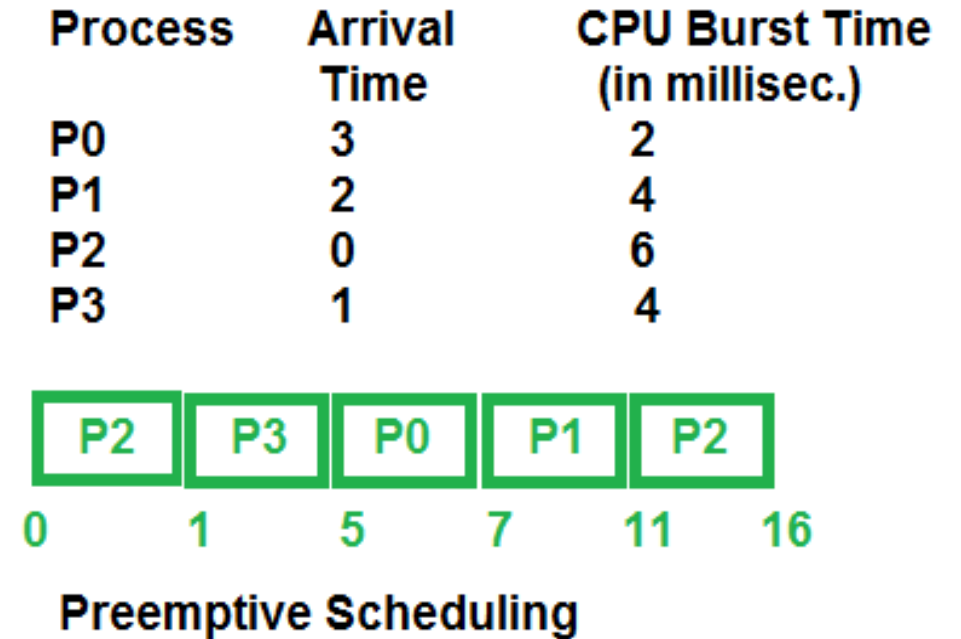
In the following example P2 is pre-empted at time 1 due to arrival of a higher priority process.

Examples:

Round Robin

Shortest Remaining Time First (SRTF)

Priority Scheduling (preemptive version)



Preemptive Scheduling

Advantages

- Prevents one process from monopolizing (Controlling) CPU
- Better average response time in multi-user systems
- Widely used in modern OS (Windows, Linux, macOS)

Disadvantages

- More complex to implement
- Higher overhead from context switching
- Risk of concurrency issues if preempted during shared resource access

Non-Preemptive Scheduling

In non-preemptive scheduling, once a process starts using the CPU, it runs until it finishes or moves to a waiting state. The OS cannot forcibly take away the CPU.

Below is the table and Gantt Chart according to the **First Come First Serve (FCFS)** Algorithm: We can notice that every process finishes execution once it gets CPU.

Examples:

1. First Come First Serve (FCFS)
2. Shortest Job First (SJF)
3. Priority Scheduling (non-preemptive version)

Process	Arrival Time	CPU Burst Time (in millisec.)
P0	3	2
P1	2	4
P2	0	6
P3	1	4



Non-Preemptive Scheduling

Non-Preemptive Scheduling

Advantages

- It is easy to implement in an operating system. It was used in Windows 3.11 and early macOS.
- It has a minimal scheduling burden.
- Less computational resources are used.

Disadvantages

- A long-running or malicious process can occupy the CPU for a long time, delaying other processes.
- Since round-robin scheduling cannot be applied, the average response time becomes high (poor), especially for short processes.

Preemptive vs Non-Preemptive Scheduling

Preemptive Scheduling	Non-Preemptive Scheduling
In this resources(CPU Cycle) are allocated to a process for a limited time.	Once resources(CPU Cycle) are allocated to a process, the process holds it till it completes its burst time or switches to waiting state
Process can be interrupted in between.	Process can not be interrupted until it terminates itself or its time is up
If a process having high priority frequently arrives in the ready queue, a low priority process may starve	If a process with a long burst time is running CPU, then later coming process with less CPU burst time may starve
It has overhead due to frequent context switching	It has minimal or no scheduling overhead.
Average process response time is less	Average process response time is high
Decisions are made by the scheduler and are based on priority and time slice allocation	Decisions are made by the process itself and the OS just follows the process's instructions
More as a process might be preempted when it was accessing a shared resource.	Less as a process is never preempted.
Examples of preemptive scheduling are Round Robin and Shortest Remaining Time First	Examples of non-preemptive scheduling are First Come First Serve and Shortest Job First

Preemptive and Non-preemptive Scheduling

CPU-scheduling decisions may take place under the following four circumstances:

1. When a process switches from the running state to the waiting state (for example, as the result of an I/O request or an invocation of `wait()` for the termination of a child process)
2. When a process switches from the running state to the ready state (for example, when an interrupt occurs)
3. When a process switches from the waiting state to the ready state (for example, at completion of I/O)
4. When a process terminates

We can say that..

- When scheduling takes place only under-circumstances 1 and 4, we say that,
- The scheduling scheme is non preemptive or cooperative. Otherwise, it is preemptive.
- Under non-preemptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases it either by terminating or by switching to the waiting state.
- Virtually all modern operating systems including Windows, macOS, Linux, and UNIX use preemptive scheduling algorithms.

Scheduling Algorithms

- First-Come, First-Serve Scheduling
- Shortest-Job-First Scheduling/ shortest-next-CPU-burst/ Shortest Process Next
 - Non Preemptive (Shortest-Job-First Scheduling)/ (Longest-Job-First Scheduling)
 - Preemptive (Shortest Remaining Time Next [SRTN]) /(Shortest Remaining Time First [SRTF])
- Round-Robin Scheduling
- Priority Scheduling
- Multilevel Queue Scheduling

1. First-Come, First-Served Scheduling

- Simplest CPU-scheduling algorithm.
- FCFS is a non-preemptive scheduling algorithm
- The process that requests the CPU first is allocated the CPU first.
- The implementation of the FCFS policy is easily managed with a FIFO queue.
- When a process enters the ready queue, its PCB is linked onto the tail of the queue. When the CPU is free, it is allocated to the process at the head of the queue.

Consider the following set of processes that arrive at time 0, with the length of the CPU burst given in milliseconds:

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

If the processes arrive in the order P_1, P_2, P_3 , and are served in FCFS order, we get the result shown in the following **Gantt chart**, which is a bar chart that illustrates a particular schedule, including the start and finish times of each of the participating processes:



Advantages of FCFS

- FCFS algorithm is simple, easy to implement into any preexisting system, and easy to understand.
- Better for processes with large burst time as there is no context switch involved between processes.
- It is a fair algorithm as priority is not involved, processes that arrive first get served first.

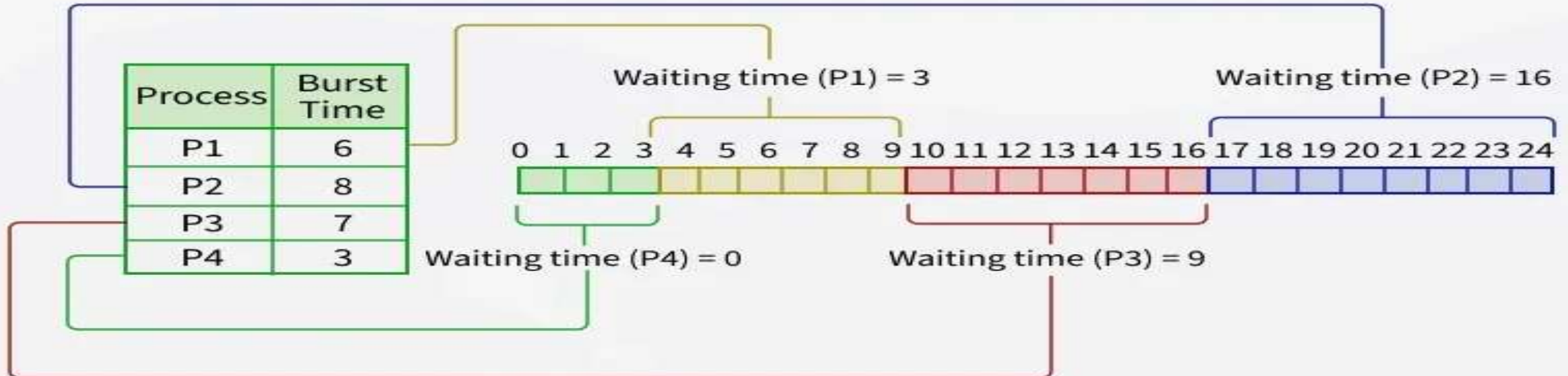
Disadvantages of FCFS

- As it is a Non-preemptive CPU Scheduling Algorithm, FCFS can result in long waiting times, especially if a long process arrives before a shorter one. This is known as the convoy effect, where shorter processes are forced to wait behind longer processes, leading to inefficient execution.
- The average waiting time in the FCFS is much higher than in the others
- Since FCFS processes tasks in the order they arrive, short jobs may have to wait a long time if they arrive after longer tasks, which leads to poor performance in systems with a mix of long and short tasks.
- Processes that are at the end of the queue, have to wait longer to finish.
- It is not suitable for time-sharing operating systems where each process should get the same amount of CPU time.

2. Shortest Job First or SJF CPU Scheduling

Shortest Job First (SJF) or Shortest Job Next (SJN) is a scheduling process that selects the waiting process with the smallest execution time to execute next. This scheduling method may or may not be preemptive. Significantly reduces the average waiting time for other processes waiting to be executed.

Shortest Job First (SJF) Scheduling Algorithm



Example: Consider the following table of arrival time and burst time for three processes P1, P2 and P3.

Process	Burst Time	Arrival Time
P1	6 ms	0 ms
P2	8 ms	2 ms
P3	3 ms	4 ms

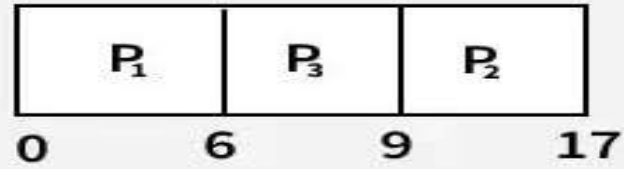
Turn Around time = Completion time - arrival time

Waiting Time = Turn around time - burst time

Step-by-Step Execution:

1. Time 0-6 (P1): P1 runs for 6 ms (total time left: 0 ms)
2. Time 6-9 (P3): P3 runs for 3 ms (total time left: 0 ms)
3. Time 9-17 (P2): P2 runs for 8 ms (total time left: 0 ms)

Gantt chart :



Shortest Job First (SJF) CPU Scheduling

Process	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT)	Waiting Time (WT)
P1	0	6	6	$6 - 0 = 6$	$6 - 6 = 0$
P2	2	8	17	$17 - 2 = 15$	$15 - 8 = 7$
P3	4	3	9	$9 - 4 = 5$	$5 - 3 = 2$

Average Turn around time = $(6 + 15 + 5)/3 = 8.6$ ms

Average waiting time = $(2 + 0 + 7)/3 = 9/3 = 3$ ms

Advantages of SJF Scheduling:

- SJF is better than the First Come First Serve (FCFS) algorithm as it reduces the average waiting time.
- SJF is generally used for long term scheduling.
- It is suitable for the jobs running in batches, where run times are already known.
- SJF is probably optimal in terms of average Turn Around Time (TAT).

Disadvantages of SJF Scheduling;

- SJF may cause very long turn-around times or starvation.
- In SJF job completion time must be known earlier.
- Many times it becomes complicated to predict the length of the upcoming CPU request.

3. Shortest Remaining Time Next (SRTN)

The pre-emptive version of Shortest Job First (SJF) scheduling is called Shortest Remaining Time First (SRTF). In SRTF, the process with the least time left to finish is selected to run. The running process continues until it finishes or a new process with a shorter remaining time arrives, ensuring the fastest finishing process always gets priority.

Example of SJF Algorithm:

Scenario 1: Processes with Same Arrival Time

Example: Consider the following table of arrival time and burst time for three processes P1, P2 and P3.

Process	Burst Time	Arrival Time
P1	6 ms	0 ms
P2	8 ms	0 ms
P3	5 ms	0 ms

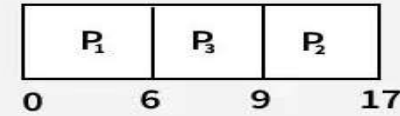
Step-by-Step Execution:

1. Time 0-5 (P3): P3 runs for 5 ms (total time left: 0 ms) as it has shortest remaining time left.
2. Time 5-11 (P1): P1 runs for 6 ms (total time left: 0 ms) as it has shortest remaining time left.
3. Time 11-19 (P2): P2 runs for 8 ms (total time left: 0 ms) as it has shortest remaining time left.

Turn Around time = Completion time - arrival time

Waiting Time = Turn around time - burst time

Gantt chart :



Shortest Job First (SJF) CPU Scheduling

Process	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT)	Waiting Time (WT)
P1	0	6	11	11-0 = 11	11-6 = 5
P2	0	8	19	19-0 = 19	19-8 = 11
P3	0	5	5	5-0 = 5	5-5 = 0

Average Turn around time = $(11 + 19 + 5)/3 = 11.6$ ms

Average waiting time = $(5 + 0 + 11)/3 = 16/3 = 5.33$ ms

Process	Burst Time	Arrival Time
P1	6 ms	0 ms
P2	3 ms	1 ms
P3	7 ms	2 ms

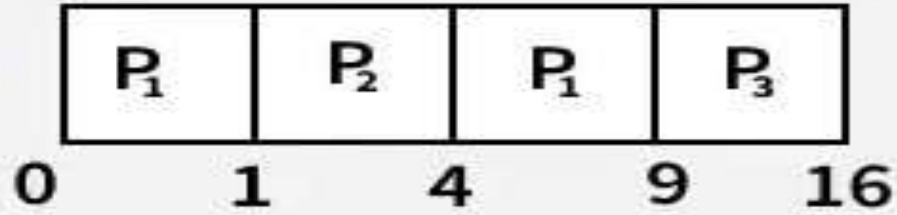
Scenario 2: Processes with Different Arrival Times

Consider the following table of arrival time and burst time for three processes P1, P2 and P3.

Step-by-Step Execution:

1. Time 0-1 (P1): P1 runs for 1 ms (total time left: 5 ms) as it has shortest remaining time left.
2. Time 1-4 (P2): P2 runs for 3 ms (total time left: 0 ms) as it has shortest remaining time left among P1 and P2.
3. Time 4-9 (P1): P1 runs for 5 ms (total time left: 0 ms) as it has shortest remaining time left among P1 and P3.
4. Time 9-16 (P3): P3 runs for 7 ms (total time left: 0 ms) as it has shortest remaining time left.

Gantt chart



Process	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turn Around Time (TAT)	Waiting Time (WT)
P1	0	6	9	$9 - 0 = 9$	$9 - 6 = 3$
P2	1	3	4	$4 - 1 = 3$	$3 - 3 = 0$
P3	2	7	16	$16 - 2 = 14$	$14 - 7 = 7$

Average Turn around time = $(9 + 14 + 3)/3 = 8.6$ ms

Average waiting time = $(3 + 0 + 7)/3 = 10/3 = 3.33$ ms

4. Round Robin (RR)

Round Robin Scheduling is a method used by operating systems to manage the execution time of multiple processes that are competing for CPU attention. It is called "round robin" because the system rotates through all the processes, allocating each of them a fixed time slice or "quantum", regardless of their priority.

Scenario 1: Processes with Same Arrival Time

Consider the following table of arrival time and burst time for three processes P1, P2 and P3 and given **Time Quantum = 2 ms**

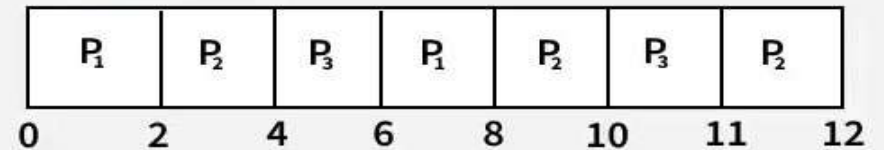
Process	Burst Time	Arrival Time
P1	4 ms	0 ms
P2	5 ms	0 ms
P3	3 ms	0 ms

Step-by-Step Execution:

1. Time 0-2 (P1): P1 runs for 2 ms (total time left: 2 ms).
2. Time 2-4 (P2): P2 runs for 2 ms (total time left: 3 ms).
3. Time 4-6 (P3): P3 runs for 2 ms (total time left: 1 ms).
4. Time 6-8 (P1): P1 finishes its last 2 ms.
5. Time 8-10 (P2): P2 runs for another 2 ms (total time left: 1 ms).
6. Time 10-11 (P3): P3 finishes its last 1 ms.
7. Time 11-12 (P2): P2 finishes its last 1 ms.

at $t = 12$

Gantt chart



Remaining time left(P₂) : 0 ms

Ready Queue :



Round Robin Scheduling Algorithm with same arrival time

- Turnaround Time = Completion Time - Arrival Time
- Waiting Time = Turnaround Time - Burst Time

Processes	AT	BT	CT	TAT	WT
P1	0	4	8	8-0 = 8	8-4 = 4
P2	0	5	12	12-0 = 12	12-5 = 7
P3	0	3	11	11-0 = 11	11-3 = 8

- Average Turn around time = $(8 + 12 + 11)/3 = 31/3 = 10.33$ ms
- Average waiting time = $(4 + 7 + 8)/3 = 19/3 = 6.33$ ms

Scenario 2: Processes with Different Arrival Times

Consider the following table of arrival time and burst time for three processes P1, P2 and P3 and given **Time Quantum = 2**

Process	Burst Time (BT)	Arrival Time (AT)
P1	5 ms	0 ms
P2	2 ms	4 ms
P3	4 ms	5 ms

Step-by-Step Execution:

1. Time 0-2 (P1 Executes):

- P1 starts execution as it arrives at 0 ms.
- Runs for 2 ms; remaining burst time = $5 - 2 = 3$ ms.
- **Ready Queue:** [P1].

2. Time 2-4 (P1 Executes Again):

- P1 continues execution since no other process has arrived yet.
- Runs for 2 ms; remaining burst time = $3 - 2 = 1$ ms.
- P2 arrive at 4 ms.
- **Ready Queue:** [P2, P1].

3. Time 4-6 (P2 Executes):

- P2 starts execution as it arrives at 4 ms.
- Runs for 2 ms; remaining burst time = $2 - 2 = 0$ ms.
- P3 arrive at 5ms
- **Ready Queue:** [P1, P3].

4. Time 6-7 (P1 Executes):

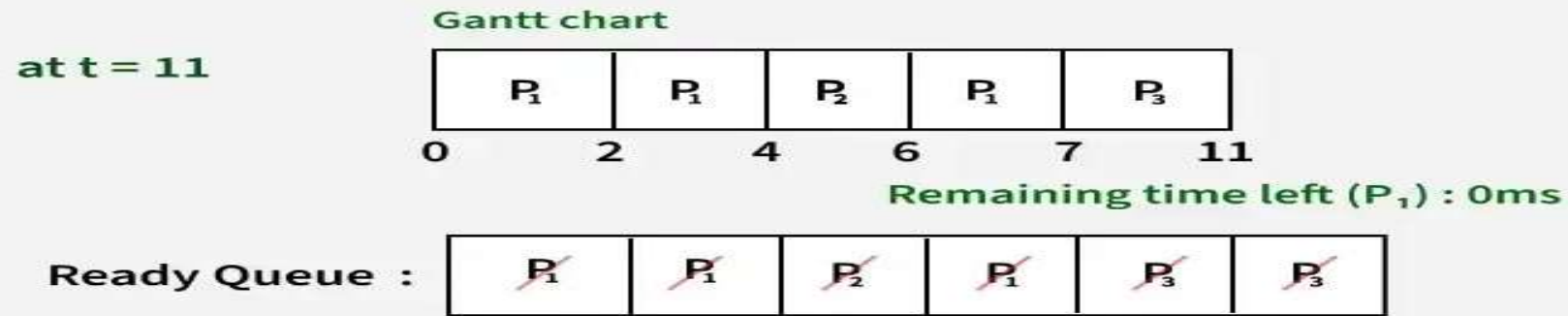
- P1 starts execution.
- Runs for 1 ms; remaining burst time = $1 - 1 = 0$ ms.
- **Ready Queue:** [P3].

5. Time 7-9 (P3 Executes):

- P3 starts execution.
- Remaining burst time = $4 - 2 = 2$ ms.
- **Ready Queue:** [P3].

6. Time 9-11 (P3 Executes Again):

- P3 resumes execution and runs for 2 ms and complete its execution
- Remaining burst time = $2 - 2 = 0$ ms.
- **Ready Queue:** [].



Round Robin Scheduling Algorithm with different Arrival Time

Process	Burst Time (BT)	Arrival Time (AT)	Process	Completion Time (CT)	Turnaround Time (TAT = CT - AT)	Waiting Time (WT = TAT - BT)
P1	5 ms	0 ms	P1	7 ms	7 ms	2 ms
P2	2 ms	4 ms	P2	6 ms	2 ms	0 ms
P3	4 ms	5 ms	P3	11 ms	6 ms	2 ms

- Average Turn around time $= 7 + 2 + 6 / 3 = 15 / 3 = 5\text{ms}$
- Average waiting time $= 2 + 0 + 2 / 3 = 1.33\text{ms}$

5. Priority Scheduling scheduling algorithm

Priority scheduling is one of the most common scheduling algorithms used by the operating system to schedule processes based on their priority. Each process is assigned a priority value based on criteria such as memory requirements, time requirements, other resource needs, or the ratio of average I/O to average CPU burst time.

The process with the highest priority is selected for execution first. If there are multiple processes sharing the same priority, they are scheduled in the order they arrived, following a First-Come, First-Served approach. The chosen process is then executed, either until completion or until it is preempted, depending on whether the scheduling is preemptive or non-preemptive.

Priority Scheduling can be implemented in two ways:

- Non-Preemptive Priority Scheduling
- Preemptive Priority Scheduling

Non-Preemptive Priority Scheduling

In Non-Preemptive Priority Scheduling, the CPU is not taken away from the running process. Even if a higher-priority process arrives, the currently running process will complete first.

Ex: A high-priority process must wait until the currently running process finishes.

Example of Non-Preemptive Priority Scheduling:

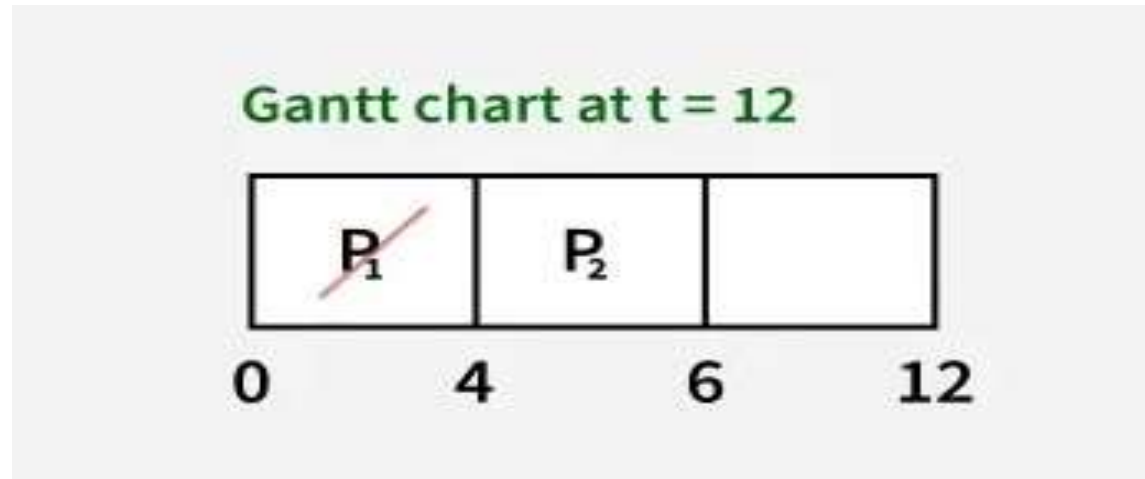
Consider the following table of arrival time and burst time for three processes P1, P2 and P3:

Note: Lower number represents higher priority.

Process	Arrival Time	Burst Time	Priority
P1	0	4	2
P2	1	2	1
P3	2	6	3

Step-by-Step Execution:

- **At Time 0:** Only P1 has arrived. P1 starts execution as it is the only available process, and it will continue executing till $t = 4$ because it is a non-preemptive approach.
- **At Time 4:** P1 finishes execution. Both P2 and P3 have arrived. Since P2 has the highest priority (Priority 1), it is selected next.
- **At Time 6:** P2 finishes execution. The only remaining process is P3, so it starts execution.
- **At Time 12:** P3 finishes execution.



Now, lets calculate average waiting time and turn around time:

Process	Arrival Time	Burst Time	Completion Time	Turnaround Time (CT - AT)	Waiting Time (TAT - BT)
P1	0	4	4	4	0
P2	1	2	6	5	3
P3	2	6	12	10	4

- Average Turnaround Time = 6.33
- Average Waiting Time = 2.33

Preemptive Priority Scheduling

In **Preemptive Priority Scheduling**, the CPU can be taken away from the currently running process if a new process with a higher priority arrives.

Ex: A low-priority process is running, and a high-priority process arrives; the CPU immediately switches to the high-priority process.

Example of Preemptive Priority Scheduling (Same Arrival Time)

Consider the following table of arrival time and burst time for three processes P1, P2 and P3:

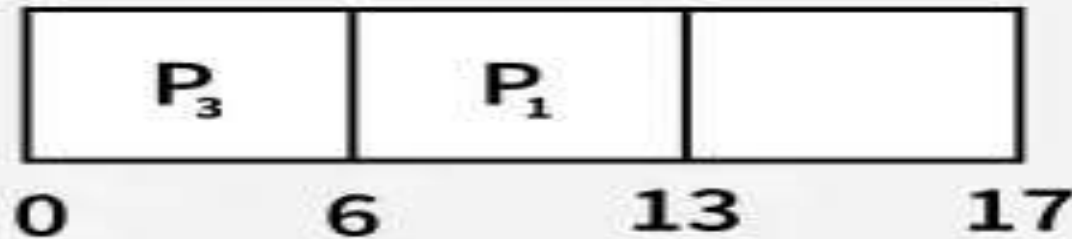
Note: Higher number represents higher priority.

Process	Arrival Time	Burst Time	Priority
P1	0	7	2
P2	0	4	1
P3	0	6	3

Step-by-Step Execution:

- **At Time 0:** All processes arrive at the same time. P3 has the highest priority (Priority 3), so it starts execution.
- **At Time 6:** P3 completes execution. Among the remaining processes, P1 (Priority 2) has a higher priority than P2, so P1 starts execution.
- **At Time 13:** P1 completes execution. The only remaining process is P2 (Priority 1), so it starts execution.
- **At Time 17:** P2 completes execution. All processes are now finished.

Gantt chart at t = 17



Now, lets calculate average waiting time and turn around time:

Process	Arrival Time	Burst Time	Completion Time	Turnaround Time (CT - AT)	Waiting Time (TAT - BT)
P1	0	7	13	13	6
P2	0	4	17	17	13
P3	0	6	6	6	0

- Average Turnaround Time = 12
- Average Waiting Time = 6.33

Example of Preemptive Priority Scheduling (Different Arrival Time)

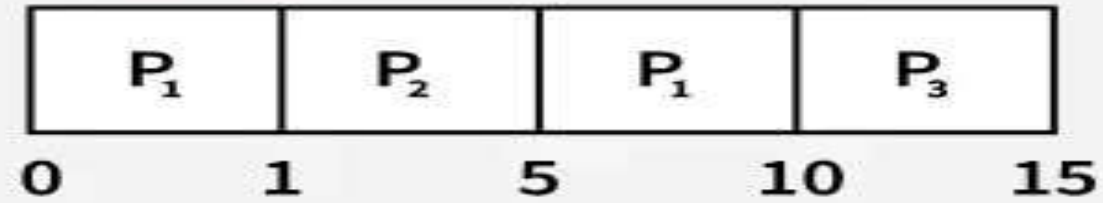
Consider the following table of arrival time and burst time for three processes P1, P2 and P3:

Process	Arrival Time	Burst Time	Priority
P1	0	6	2
P2	1	4	3
P3	2	5	1

Step-by-Step Execution:

- **At Time 0:** Only P1 has arrived, so it starts execution.
- **At Time 1:** P2 arrives with a higher priority (Priority 3) than P1. P1 is preempted, and P2 starts execution.
- **At Time 5:** P2 completes execution. Both P1 and P3 are available. P1 has the higher priority (Priority 2), so it starts execution.
- **At Time 10:** P1 completes execution. P3 resumes execution to finish its remaining burst time.
- **At Time 15:** P3 completes execution. All processes are now finished.

Gantt chart at t = 15



Now, let's calculate average waiting time and turn around time:

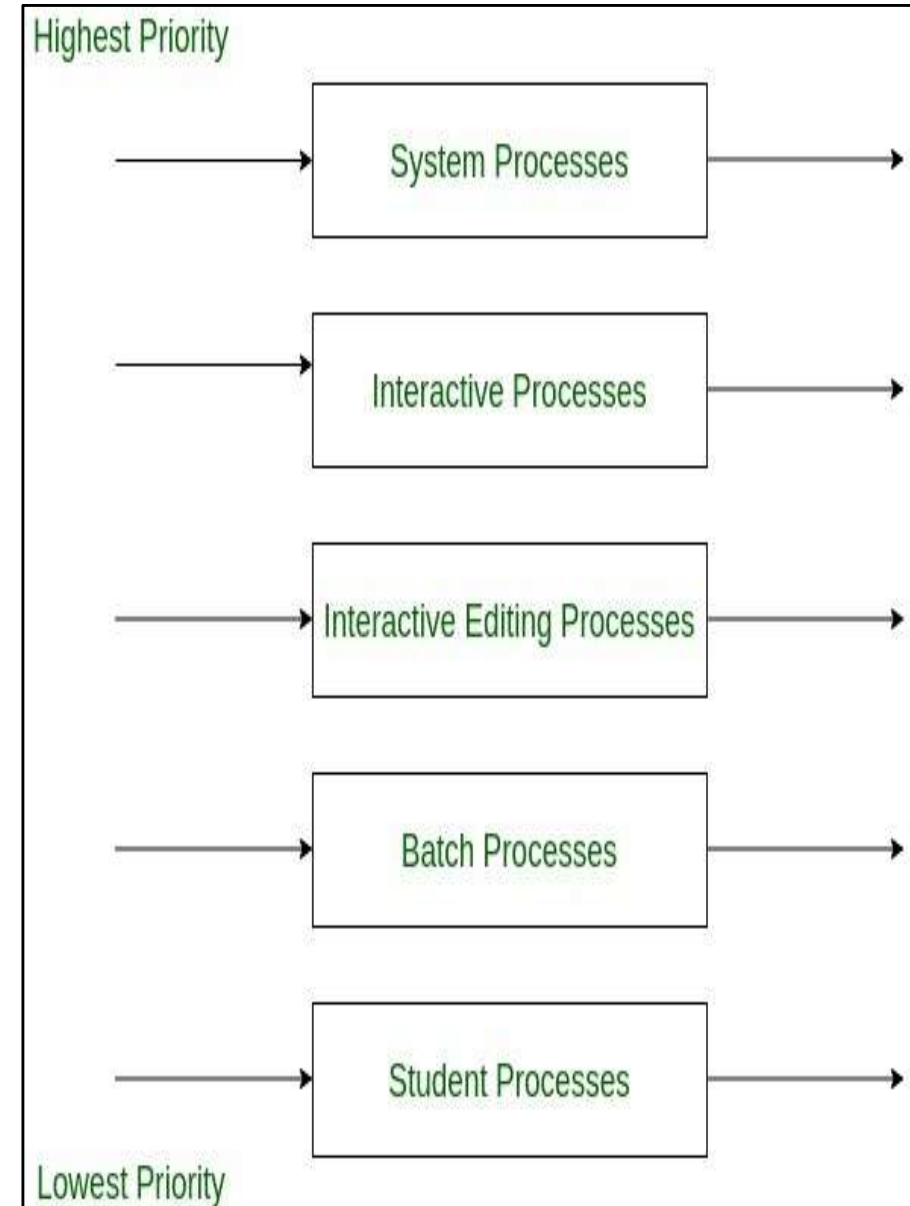
Processes	Arrival Time	Burst Time	Completion Time	Turnaround Time (CT - AT)	Waiting Time (TAT - BT)
P1	0	6	10	10	4
P2	1	4	5	4	0
P3	2	5	15	13	8

- Average Turnaround Time = 9
- Average Waiting Time = 4

6. Multilevel Queue (MLQ) CPU Scheduling

Multilevel Queue Scheduling divides processes into multiple queues arranged in a hierarchy.

- Each queue has its own priority level and process type.
- Different scheduling algorithms can be applied to different queues.
- Once assigned, a process is permanently mapped to a specific queue.
- Assignment is usually based on factors like priority, resources, or process characteristics.
- **Ready Queue** is divided into separate queues for each class of processes. For example, let us take five different types of processes: System processes, Interactive processes, Interactive Editing processes, Batch Processes and Student processes. All five processes have their own queue. Now, look at the below figure.



- **System Process:** The OS has its process to execute, which is referred to as the System Process. System processes are the processes created and used by the Operating System itself to perform system-related tasks.

Example: Processes related to memory management, device management, and other OS services.

- **Interactive Process:** It is a process in which the user interacts with the program continuously in the same way. A process that interacts directly with the user and requires quick responses from the operating system.

Examples: Web browser , Text editor , Command-line terminal , Chat application

- **Interactive Editing Process:** An Interactive Editing Process is a process where the user continuously interacts with a program while editing or modifying data.

Example: Editing a document in MS Word, Writing and editing code, Editing a photo

- **Batch Process:** Batch processing is an operating system feature that collects programs and data into a batch before processing starts.

Example: Generating reports, Large data processing

- **User Process:** A User Process Queue contains processes that are created and executed for the user's applications or tasks.

Examples: Opening a document, Running a calculator program, Running a user-created program

6. Multilevel Queue (MLQ) CPU Scheduling	
Advantages	Disadvantages
• Separates processes (e.g., system, interactive, batch) into distinct queues, making management more organized.	• If high-priority queues are underloaded, CPU may stay idle while lower-priority processes wait.
• Since queue priorities are fixed, system administrators can easily predict how processes will be scheduled.	• Once placed in a queue, processes cannot move between queues, leading to inefficiency if workload type changes.
• Interactive and system processes get higher priority, leading to improved response times for user-centric tasks.	• Handling multiple queues requires more bookkeeping and resources.
• Suitable for systems handling a wide variety of process types, as multiple queues can be designed for different workloads.	• Some queues may dominate CPU time while others rarely get a chance
• Ensures critical processes are not delayed behind non-essential tasks.	• In real-world workloads where process behavior changes, fixed queues reduce flexibility and adaptability.

Advantages of Multilevel Queue CPU Scheduling:

- The processes are permanently assigned to the queue, so it has advantage of low scheduling overhead.

Disadvantages of Multilevel Queue CPU Scheduling:

- Some processes may starve for CPU if some higher priority queues are never becoming empty.
- It is inflexible in nature.

DEADLOCK

Deadlock System Model

Deadlock is a situation in computing where two or more processes are unable to proceed because each is waiting for the other to release resources. Key concepts include mutual exclusion, resource holding, circular wait, and no preemption. Consider a practical example when two trains are coming toward each other on the same track and there is only one track, none of the trains can move once they are in front of each other.

A deadlock occurs when a set of processes is stalled (Halted) because each process is holding a resource and waiting for another process to acquire another resource. In the diagram below, for example, Process 1 is holding Resource 1 while Process 2 acquires Resource 2, and Process 2 is waiting for Resource 1.

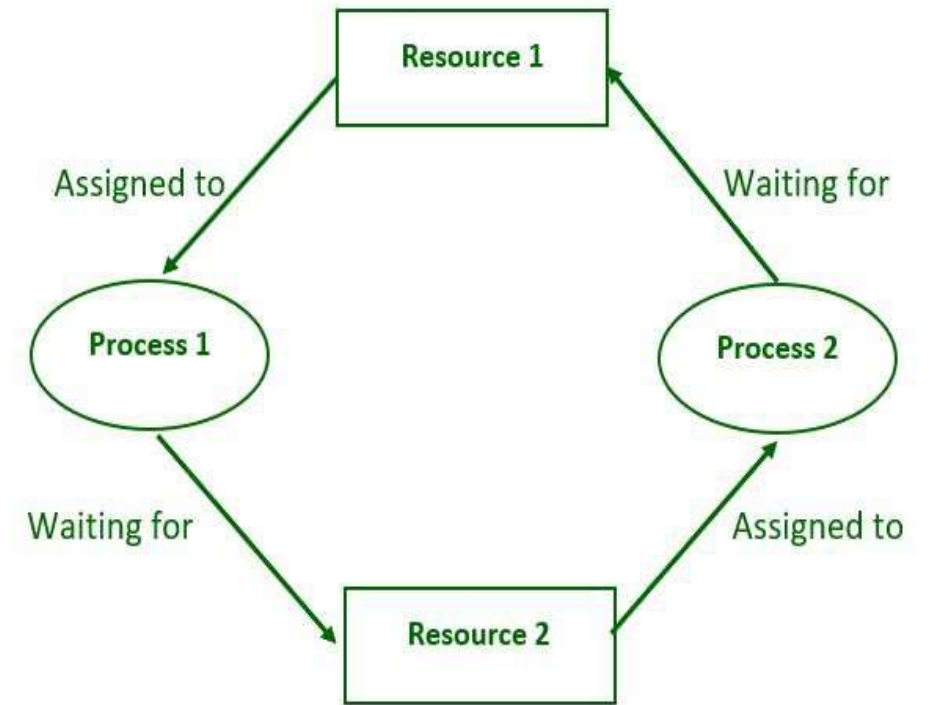


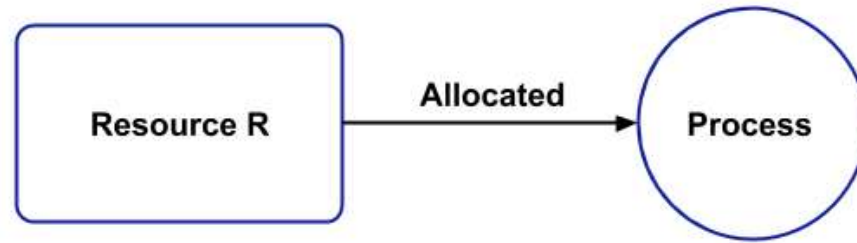
Figure: Deadlock in Operating system

Necessary Conditions of a Deadlock

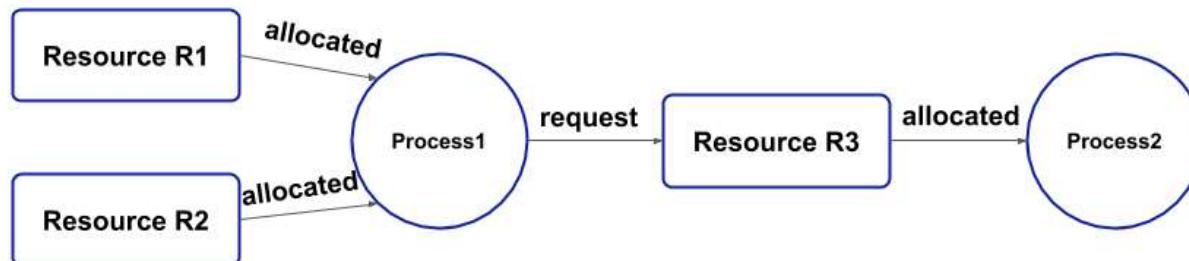
There are four different conditions that result in Deadlock. These four conditions are also known as Coffman conditions. Let's look at them one by one.

1. Mutual Exclusion
2. Hold and Wait
3. No Pre Emption
4. Circular Wait

1. Mutual Exclusion: A resource can be held by only one process at a time. In other words, if a process P1 is using some resource R at a particular instant of time, then some other process P2 can't hold or use the same resource R at that particular instant of time. The process P2 can make a request for that resource R but it can't use that resource simultaneously with process P1.

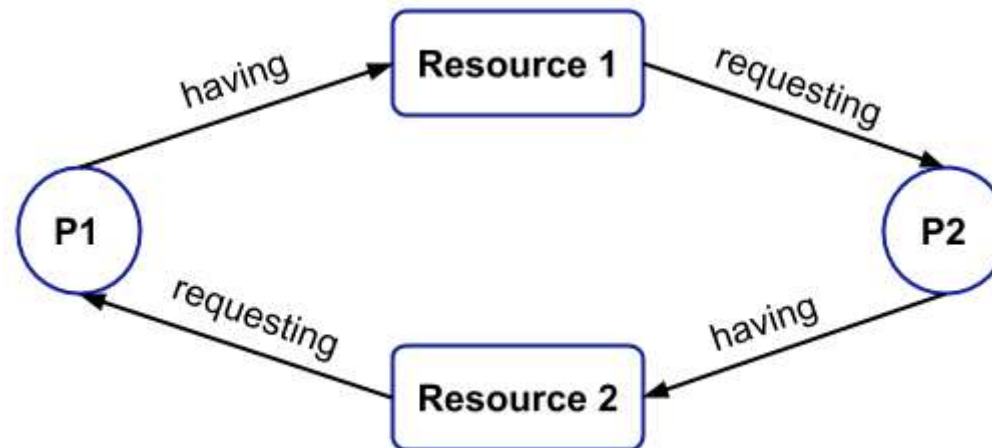


2. Hold and Wait: A process can hold a number of resources at a time and at the same time, it can request for other resources that are being held by some other process. For example, a process P1 can hold two resources R1 and R2 and at the same time, it can request some resource R3 that is currently held by process P2.



3. No preemption: A resource can't be preempted from the process by another process, forcefully. For example, if a process P1 is using some resource R, then some other process P2 can't forcefully take that resource. The process P2 can request for the resource R and can wait for that resource to be freed by the process P1.

4. Circular Wait: Circular wait is a condition when the first process is waiting for the resource held by the second process, the second process is waiting for the resource held by the third process, and so on. At last, the last process is waiting for the resource held by the first process. So, every process is waiting for each other to release the resource and no one is releasing their own resource. Everyone is waiting here for getting the resource. This is called a circular wait.

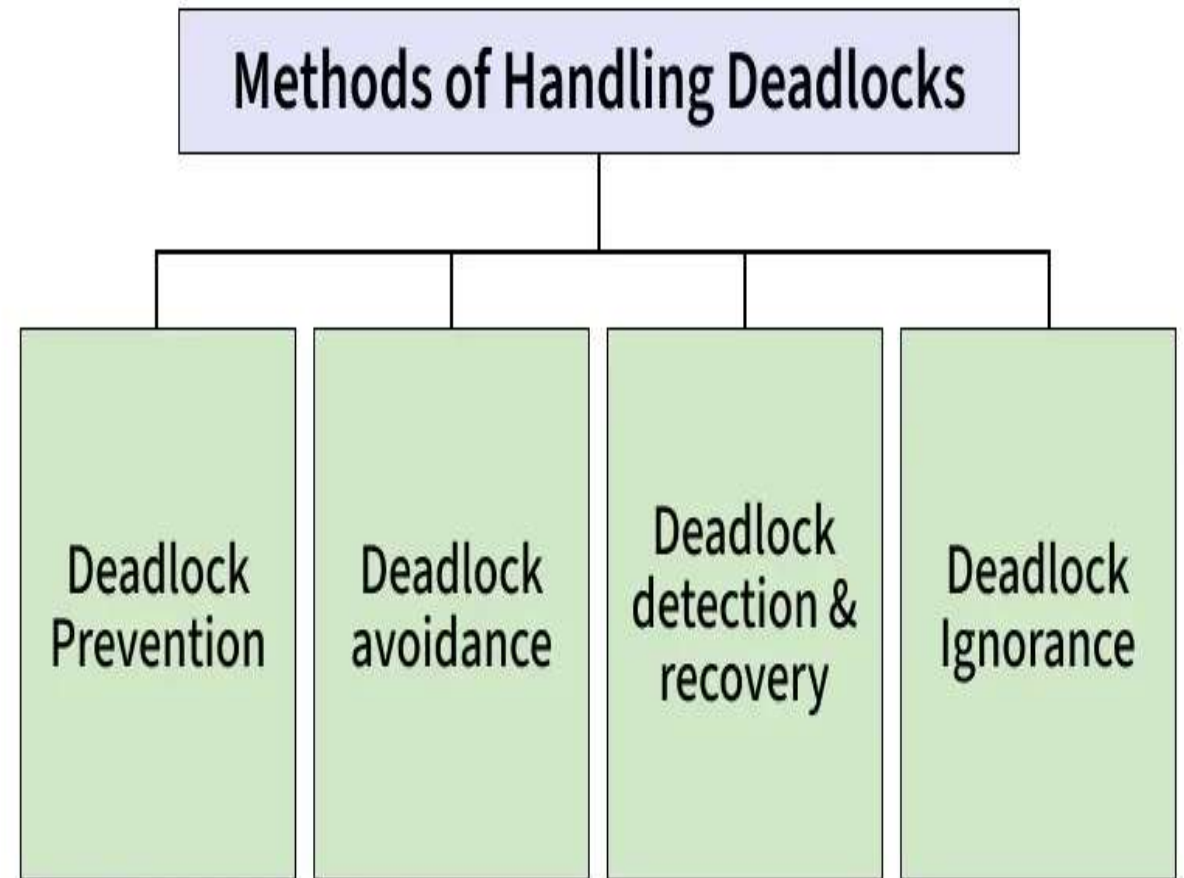


Deadlock Handling

Deadlock handling methods are strategies used in operating systems to ensure processes do not remain permanently blocked, maintaining smooth execution and system reliability.

Methods of Handling Deadlocks

There are four approaches to dealing with deadlocks.



Deadlock Prevention

Deadlock prevention is a strategy used in computer systems to ensure that different processes can run smoothly without getting stuck waiting for each other forever. Think of it like a traffic system where cars (processes) must move through intersections (resources) without getting into a gridlock.

Deadlock can only happen if all four of the following conditions are met simultaneously:

1. Mutual Exclusion
2. Hold and Wait
3. No Preemption
4. Circular Wait

1. Eliminate Mutual Exclusion

- We need to make the resources sharable with the help of algorithms like round robin.

2. Eliminate Hold and Wait

Hold and wait is a condition in which a process holds one resource while simultaneously waiting for another resource that is being held by a different process. The process cannot continue until it gets all the required resources.

There are two ways to eliminate hold and wait:

By eliminating wait:

The process specifies the resources it requires in advance so that it does not have to wait for allocation after execution starts.

For Example, Process1 declares in advance that it requires both Resource1 and Resource2.

By eliminating hold:

The process has to release all resources it is currently holding before making a new request. For Example: Process1 must release Resource2 and Resource3 before requesting Resource1

3. Eliminate No Preemption

No preemption means resources can't be taken away once allocated. To prevent this:

Processes must release resources voluntarily(freely): A process gives up resources once it finishes using them.

Avoid partial allocation: If a process requests resources that are unavailable, it must release all currently held resources and wait until all required resources are free.

4. Eliminate Circular Wait

Circular wait happens when processes form a cycle, each waiting for a resource held by the next. To prevent this:

- Impose a strict ordering on resources.
- Assign each resource a unique number.
- Processes can only request resources in increasing order of their numbers.

Deadlock Avoidance

Deadlock Avoidance is a method where the operating system makes decisions dynamically to ensure the system never enters an unsafe state, thereby avoiding the possibility of deadlock. This is usually done using algorithms like the Banker's Algorithm.

Deadlock avoidance mainly relies on algorithms that check resource allocation before making decisions.

The two common types are:

Banker's Algorithm: Used when multiple instances of resources exist. It simulates allocation and ensures that the system remains in a safe state before granting resources.

Resource Allocation Graph (RAG) Algorithm: Used when each resource has only one instance. It checks for cycles in the graph to avoid unsafe states.

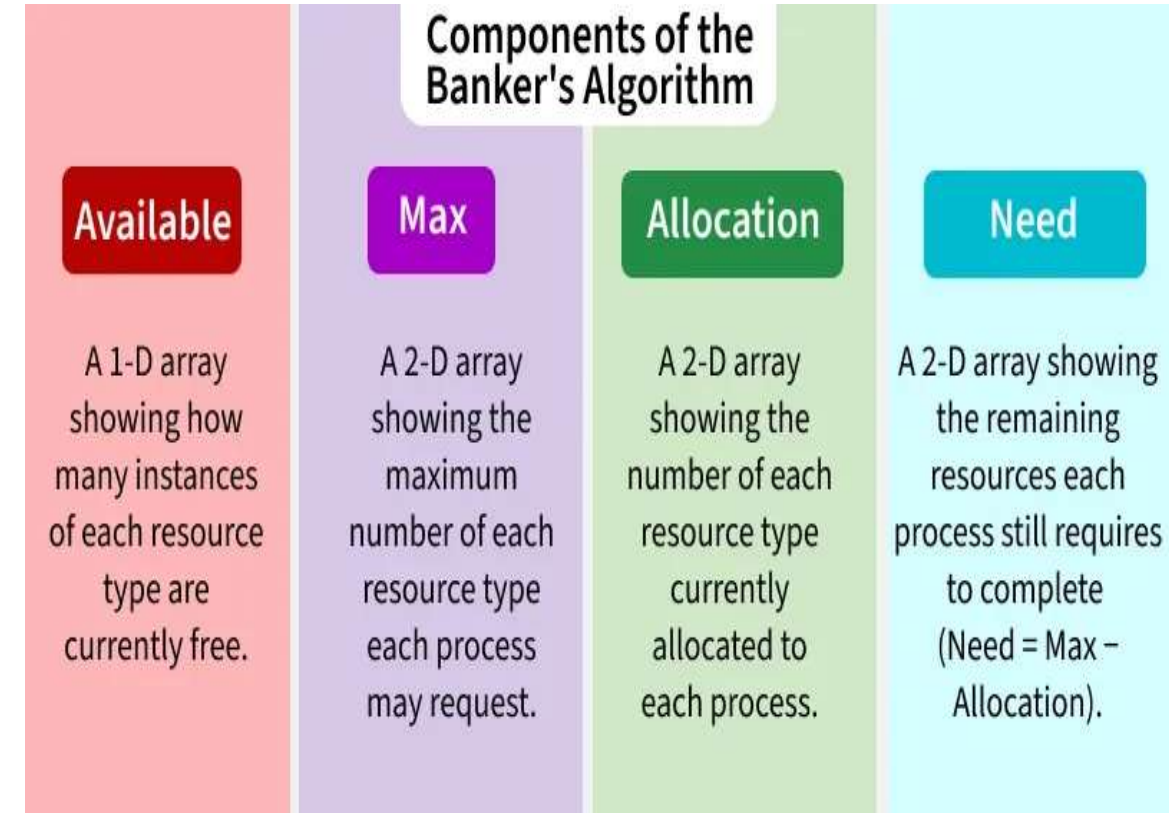
Banker's Algorithm

Banker's Algorithm is a resource allocation and deadlock avoidance algorithm used in operating systems. It checks whether granting resource requests keeps the system in a safe state, where at least one safe sequence exists in which all processes can complete.

- It avoids deadlock by ensuring that resource allocations keep the system in a safe state.
- It keeps track of the resources each process needs and the resources currently available.

Components of the Banker's Algorithm

The following Data structures are used to implement the Banker's Algorithm as shown in figure:



Key Concepts in Banker's Algorithm

- **Safe State:** There exists at least one sequence of processes such that each process can obtain the needed resources, complete its execution, release its resources, and thus allow other processes to eventually complete without entering a deadlock.
- **Unsafe State:** Even though the system can still allocate resources to some processes, there is no guarantee that all processes can finish without potentially causing a deadlock. An unsafe state does not necessarily mean that a deadlock has already occurred. It means the system cannot guarantee that all processes can complete without deadlock.

Thank You!